

# RTL Design Sherpa

# Delta Micro-Architecture Specification 1.0

**October 12, 2026**

# Table of Contents

1 Delta Mas Index....12

2 Delta Hardware Architecture Specification Index....13

2.1 Overview....13

2.2 Related Modules....14

2.3 Navigation....15

2.3.1 Front Matter....15

2.3.2 Chapter 1: Introduction....15

2.3.3 Chapter 2: System Overview....15

2.3.4 Chapter 3: Architecture....15

2.3.5 Chapter 4: Interfaces....15

2.3.6 Chapter 5: Performance....15

2.3.7 Chapter 6: Integration....15

3 Delta: AXI-Stream Crossbar Generator - Product Requirements Document....17

3.1 Executive Summary....18

3.2 1. Project Goals....19

3.2.1 1.1 Primary Goals....19

3.2.2 1.2 Non-Goals....19

3.3 2. Architecture....20

3.3.1 2.1 Flat Crossbar Topology....20

3.3.2 2.2 Tree Topology (retired 2026-10-12 — never implemented in the framework generator)....20

3.4 3. Functional Requirements....22

3.4.1 3.1 Code Generation....22

3.4.2 3.2 Performance Modeling....22
3.4.3 3.3 Protocol Compliance....22
3.5 4. Non-Functional Requirements....24
3.5.1 4.1 Performance Targets....24
3.5.2 4.2 Resource Targets....24
3.5.3 4.3 Code Quality....24
3.6 5. Interface Specifications....25
3.6.1 5.1 AXI-Stream Master Interface (S_AXIS)....25
3.6.2 5.2 AXI-Stream Slave Interface (M_AXIS)....25
3.6.3 5.3 Clock and Reset....26
3.7 6. Key Design Decisions....27
3.7.1 6.1 Why AXI-Stream vs AXI4?....27
3.7.2 6.2 Why Round-Robin vs Fixed Priority?....27
3.7.3 6.3 Why Flat AND Tree Topologies?....27
3.7.4 6.4 Why Python Generator vs Manual RTL?....27
3.8 7. Comparison to APB Crossbar....28
3.8.1 7.1 Similarities (~95% Code Reuse)....28
3.8.2 7.2 Differences....28
3.8.3 7.3 Migration Effort....28
3.9 8. Use Cases....29
3.9.1 8.1 Primary: RISC Cores + DSP Array....29
3.9.2 8.2 Secondary: Educational Demonstration....29
3.9.3 8.3 Future: Integration with RAPIDS....29
3.10 9. Verification Strategy....30

3.10.1 9.1 Generator Verification....30
3.10.2 9.2 RTL Verification....30
3.10.3 9.3 Model Validation....30
3.11 10. Documentation Requirements....31
3.11.1 10.1 Specifications....31
3.11.2 10.2 Code Documentation....31
3.12 11. Success Metrics....32
3.12.1 11.1 Functional Metrics....32
3.12.2 11.2 Performance Metrics....32
3.12.3 11.3 Quality Metrics....32
3.13 12. Timeline and Milestones....33
3.13.1 Week 1: Generator and Models....33
3.13.2 Week 2: RTL and Verification....33
3.13.3 Week 3: Integration and Documentation....33
3.14 13. Open Questions....34
3.15 14. Revision History....35
3.16 16. Documentation....36
3.17 Appendix A: Glossary....37
4 Claude Code Guide: Delta (AXI-Stream Crossbar Generator)....38
4.1 Quick Context....39
4.2 How It Actually Works (verified against rtl/delta_axis_flat_4x16.sv)....40
4.3 Repository Map....41
4.4 Critical Rules....42
4.5 Common Tasks....43

5 Document Information....44
5.1 Delta Micro-Architecture Specification....44
5.2 Revision History....45
5.3 Document Purpose....46
5.4 Intended Audience....47
5.5 Related Documents....48
6 Overview....49
6.1 What This Document Describes....49
6.2 Generated Module Structure....50
6.3 Key Micro-Architectural Decisions....51
6.4 Block Diagram....52
6.4.1 Figure 1.1: Delta Micro-Architecture Block Diagram....52
6.5 Reference RTL....55
7 Request Decode....56
7.1 Purpose....56
7.2 Implementation....57
7.3 TDEST Range Check....58
7.4 Relationship to Arbitration....59
7.5 Notes....60
8 Per-Slave Round-Robin Arbiter....61
8.1 State Variables....61
8.2 FSM Behavior....62
8.2.1 Idle State (packet_active[s] == 0)....62
8.2.2 Active State (packet_active[s] == 1)....62

8.3 Round-Robin Pseudocode ..... 63
8.4 Fairness ..... 64
8.5 Reset Behavior ..... 65
9 Crossbar Multiplexer ..... 66
9.1 Purpose ..... 66
9.2 Implementation ..... 67
9.3 Signal Selection ..... 68
9.4 Default Values ..... 69
9.5 Timing ..... 70
10 TREADY Steering ..... 71
10.1 Purpose ..... 71
10.2 Implementation ..... 72
10.3 Behavior ..... 73
10.4 Deadlock Avoidance ..... 74
11 TWAKEUP Steering (AMBA5) ..... 75
11.1 Purpose ..... 75
11.2 Implementation ..... 76
11.3 Semantics ..... 77
11.4 Integration Notes ..... 78
12 Packet Counters ..... 79
12.1 Purpose ..... 79
12.2 Implementation ..... 80
12.3 Behavior ..... 81
12.4 Disabling Counters ..... 82

13 Assertions....83
13.1 Overview....83
13.2 One-Hot Grant Check....84
13.3 TDEST Range Check....85
13.4 Enabling Assertions....86
13.5 Notes....87
14 Multi-Master Contention....88
14.1 Scenario....88
14.2 Cycle Trace....89
14.3 Observations....91
14.4 Same-Cycle Release and Re-Arbitration....92
15 Packet Interleave Prevention....93
15.1 Rule....93
15.2 Mechanism....94
15.3 Why It Matters....95
15.4 Example: Attempted Interleave....96
15.5 Corner Cases....97
16 Module Structure....98
16.1 Generated File....98
16.2 Module Header....99
16.2.1 Base Flavor Header....99
16.2.2 Rich Flavor Additions....99
16.3 Internal Organization....100
16.4 Filelist....101

17 Parameters and Logic Implications....102
17.1 Parameter Summary....102
17.2 Width Calculations....103
17.3 Parameter Interaction....104
17.4 RTL Constants....105
18 Test Strategy....106
18.1 Current Status....106
18.2 Recommended Test Plan....107
18.2.1 1. Routing Coverage....107
18.2.2 2. Arbitration Fairness....107
18.2.3 3. Packet Atomicity....107
18.2.4 4. Backpressure....107
18.2.5 5. Reset....107
18.2.6 6. Rich Flavor....107
18.2.7 7. Formal Assertions....107
18.3 Lint and Generation Checks....108

# List of Figures

Figure 1.1: Delta Micro-Architecture Block Diagram.....52

# List of Tables

Table 1: Table 1.1: Key Micro-Architectural Decisions....51
Table 2: Table 2.1: Arbiter State Variables....61
Table 3: Table 3.1: Contention Trace....89
Table 4: Table 3.2: Interleave Prevention....96
Table 5: Table 4.1: Parameter-to-Logic Mapping....102

# List of Waveforms

*No waveforms in this document.*

# 1 Delta Mas Index

**Generated:** 2026-10-12

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 2 Delta Hardware Architecture Specification Index

## 2.1 Overview

**Version:** 1.0 **Date:** 2026-10-12 **Purpose:** High-level hardware architecture specification for the Delta AXI4-Stream crossbar generator

---

## 2.2 Related Modules

- **Delta MAS** - Micro-Architecture Specification (detailed block-level)
- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI development guide
- **QUICK_START.md** - Hands-on generator guide

## 2.3 Navigation

**Note:** Every chapter below is one source file; the document build assembles the spec from these links.

### 2.3.1 Front Matter

- Document Information

### 2.3.2 Chapter 1: Introduction

- Purpose and Scope
- Document Conventions
- Definitions and Acronyms

### 2.3.3 Chapter 2: System Overview

- Use Cases
- Key Features
- System Context

### 2.3.4 Chapter 3: Architecture

- Block Diagram
- Data Flow
- Protocol Support

### 2.3.5 Chapter 4: Interfaces

- AXI-Stream Base Interface
- AXI-Stream Rich Interface (TSTRB + AMBA5)
- Clock and Reset

### 2.3.6 Chapter 5: Performance

- Throughput Characteristics
- Latency Analysis
- Resources

### 2.3.7 Chapter 6: Integration

- System Requirements
- Parameter Configuration
- Verification Strategy
- Synthesis and Implementation

---

**Last Updated:** 2026-10-12 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 3 Delta: AXI-Stream Crossbar Generator - Product Requirements Document

**Project:** Delta **Version:** 1.0 **Date:** 2025-10-18 **Status:** Active Development

---

## 3.1 Executive Summary

**Delta** is an AXI-Stream crossbar generator that produces parameterized RTL for routing data between multiple masters and multiple slaves. The name “Delta” follows the water/river theme (like RAPIDS) - deltas are where rivers split into multiple branches, analogous to crossbar routing.

**Key Features:** - Python-based RTL generation (similar to APB crossbar automation) - Single topology support: flat crossbar (tree topology retired 2026-10-12 — see QUICK_START.md) - Performance modeling (analytical + simulation) - Complete AXI-Stream protocol compliance - Educational focus with rigorous specifications

---

## 3.2 1. Project Goals

### 3.2.1 1.1 Primary Goals

1. **Code Generation Excellence**
   - Python generator produces clean, parameterized SystemVerilog
   - Reuses patterns from existing APB crossbar automation (~95% code reuse)
   - Single tool generates multiple topologies
2. **Performance Rigor**
   - Analytical models (closed-form latency/throughput)
   - Discrete event simulation (SimPy)
   - Resource estimation
   - Validation against RTL synthesis
3. **Educational Value**
   - Demonstrates specification-driven design
   - Shows code generation techniques
   - Compares topology trade-offs
   - Suitable for instruction on GitHub
4. **Integration Ready**
   - Works with existing CocoTBFramework
   - Compatible with RAPIDS subsystem
   - Supports RISC core + DSP array use case

### 3.2.2 1.2 Non-Goals

- Vendor-specific IP (using custom generators only)
- Protocol conversion (AXIS only, not AXI4/APB)
- Advanced routing algorithms (static TDEST-based only)
- Dynamic reconfiguration

---

## 3.3 2. Architecture

### 3.3.1 2.1 Flat Crossbar Topology

**Description:** Full MxN crossbar with per-slave arbiters

**Characteristics:** - **Latency:** 2 cycles (arbitration + output register) - **Throughput:** High aggregate (each slave independent) - **Resources:** ~1,920 LUTs for 4x16 @ 64-bit - **Best For:** Production systems requiring low latency

**Block Diagram:**

```
Masters (M)          Arbiters (N)         Slaves (N)

M0 ------+----------+-[Arbiter_S0]----- S0
         |          |
M1 ------+          +-[Arbiter_S1]----- S1
         |          |
M2 ------+          +-[Arbiter_S2]----- S2
         |          |
M3 ------+          +-[Arbiter_S15]---- S15

Request Matrix      Grant Matrix
(decode TDEST)      (round-robin)
```

### 3.3.2 2.2 Tree Topology (retired 2026-10-12 — never implemented in the framework generator)

**Description:** Hierarchical composition of 1:2 splitters and 2:1 mergers

**Characteristics:** - **Latency:** 4-6 cycles (multi-stage pipeline) - **Throughput:** Lower aggregate (bottleneck at root) - **Resources:** ~1,600 LUTs for 4x16 @ 64-bit (fewer LUTs but more instances) - **Best For:** Educational examples, demonstrating modularity

**Block Diagram:**

```
Masters     Mergers        Splitters       Slaves

M0 --+
     +-[2:1]--[1:2]--+-[1:2]--+- S0
M1 --+               |         +- S1
                     |
M2 --+               +-[1:2]--+- S2
     +-[2:1]--------+         +- S3
M3 --+

Tree depth: log2(N) stages
```

## 3.4 3. Functional Requirements

### 3.4.1 3.1 Code Generation

**REQ-GEN-001:** Python generator shall produce SystemVerilog RTL - **Input:** Command-line parameters (masters, slaves, data width, topology) - **Output:** Synthesizable SystemVerilog modules - **Verification:** Verilator lint clean, synthesis clean

**REQ-GEN-002:** Generator shall support flat crossbar topology (tree topology retired 2026-10-12 — see QUICK_START.md) **REQ-GEN-003:** Generator shall support optional TSTRB byte qualifiers (`--strb-width N`; default 0 = base signal set) so generated crossbars port-match `rtl/amba/axis4/*` (added 2026-10-12). **REQ-GEN-004:** Generator shall support optional AMBA 5 sidebands (`--amba5`): TWAKEUP steering — TDEST-decoded OR per slave, valid before TVALID — and TPARITY (`--parity-width`, default DATA_WIDTH/8), matching `rtl/amba/axis5/*` (added 2026-10-12). Both options default off; default output is byte-identical to pre-2026-10-12 generations. - **Flat:** Single crossbar module - **Tree:** Hierarchical node composition - **Both:** ~~Generate both variants with `--topology both`~~ (retired 2026-10-12; flat only)

**REQ-GEN-003:** Generated RTL shall be parameterized - NUM_MASTERS, NUM_SLAVES (up to 32x256) - DATA_WIDTH (8, 16, 32, 64, 128, 256, 512, 1024 bits) - DEST_WIDTH, ID_WIDTH (auto-calculated)

**REQ-GEN-004:** Generator shall follow APB crossbar patterns - ~95% code reuse from APB automation - Same request generation, arbitration, mux patterns - Signal name mapping (pclk->aclk, psel->tvalid, etc.)

### 3.4.2 3.2 Performance Modeling

**REQ-MODEL-001:** Analytical model shall provide closed-form results - Latency calculation (cycles and nanoseconds) - Throughput estimation (transfers/cycle, Gbps) - Resource estimation (LUTs, FFs)

**REQ-MODEL-002:** Simulation model shall use discrete event simulation - SimPy-based cycle-accurate model - Statistical analysis (mean, percentiles) - Traffic pattern support (uniform, hotspot, localized)

**REQ-MODEL-003:** Models shall be validated against RTL - Compare analytical vs simulation results - Validate against synthesis reports - Document discrepancies

### 3.4.3 3.3 Protocol Compliance

**REQ-PROTO-001:** Generated RTL shall comply with AXI-Stream specification - TVALID/TREADY handshaking - TLAST packet boundaries - TDEST, TID, TUSER sideband signals

**REQ-PROTO-002:** Arbitration shall provide packet atomicity - Grant locked until TLAST - No packet interleaving - Fair round-robin arbitration

**REQ-PROTO-003:** Backpressure shall propagate correctly - TREADY from slave to granted master - No deadlocks - No data loss

---

## 3.5 4. Non-Functional Requirements

### 3.5.1 4.1 Performance Targets

**NFR-PERF-001:** Flat crossbar latency <= 2 cycles **NFR-PERF-002:** Flat crossbar Fmax >= 300 MHz (UltraScale+) **NFR-PERF-003:** ~~Tree topology latency <= 6 cycles~~ (retired with tree topology, 2026-10-12) **NFR-PERF-004:** Throughput >= 0.7 transfers/cycle under mixed traffic

### 3.5.2 4.2 Resource Targets

**NFR-RES-001:** Flat 4x16 @ 64-bit <= 2,500 LUTs **NFR-RES-002:** Tree 4x16 @ 64-bit <= 2,000 LUTs **NFR-RES-003:** No BRAM usage (pure logic implementation)

### 3.5.3 4.3 Code Quality

**NFR-QUAL-001:** Generated RTL shall pass Verilator lint **NFR-QUAL-002:** Generator code shall follow PEP 8 style **NFR-QUAL-003:** Comprehensive inline documentation **NFR-QUAL-004:** Assertions for formal verification

---

## 3.6 5. Interface Specifications

### 3.6.1 5.1 AXI-Stream Master Interface (S_AXIS)

| Signal | Width | Direction | Description |
|---|---|---|---|
| s_axis_tdata[m] | DATA_WIDTH | Input | Data payload |
| s_axis_tvalid[m] | 1 | Input | Valid indicator |
| s_axis_tready[m] | 1 | Output | Ready (backpressure) |
| s_axis_tlast[m] | 1 | Input | Packet boundary |
| s_axis_tdest[m] | DEST_WIDTH | Input | Target slave ID |
| s_axis_tid[m] | ID_WIDTH | Input | Transaction ID |
| s_axis_tuser[m] | USER_WIDTH | Input | User sideband |

**Array:** `[NUM_MASTERS]` (one per master)

### 3.6.2 5.2 AXI-Stream Slave Interface (M_AXIS)

| Signal | Width | Direction | Description |
|---|---|---|---|
| m_axis_tdata[s] | DATA_WIDTH | Output | Data payload |
| m_axis_tvalid[s] | 1 | Output | Valid indicator |
| m_axis_tready[s] | 1 | Input | Ready (backpressure) |
| m_axis_tlast[s] | 1 | Output | Packet boundary |
| m_axis_tdest[s] | DEST_WIDTH | Output | Target slave ID (pass-through) |
| m_axis_tid[s] | ID_WIDTH | Output | Transaction ID (pass-through) |
| m_axis_tuser[s] | USER_WIDTH | Output | User sideband (pass-through) |

**Array:** `[NUM_SLAVES]` (one per slave)

### 3.6.3 5.3 Clock and Reset

| Signal | Width | Direction | Description |
|---|---|---|---|
| aclk | 1 | Input | Clock |
| aresetn | 1 | Input | Active-low reset |

## 3.7 6. Key Design Decisions

### 3.7.1 6.1 Why AXI-Stream vs AXI4?

**Decision:** Use AXI-Stream instead of full AXI4

**Rationale:** - Streaming workloads (RISC cores, DSP arrays) - Simpler protocol (no address phases, burst management) - Better fit for compute fabrics - Still allows future upgrade to AXI4 if needed

### 3.7.2 6.2 Why Round-Robin vs Fixed Priority?

**Decision:** Default to round-robin arbitration

**Rationale:** - Fair bandwidth allocation - No starvation - Suitable for general-purpose compute - Can add fixed-priority mode later if needed

### 3.7.3 6.3 Why Flat AND Tree Topologies?

**Decision:** Support both, not either/or

**Rationale:** - Flat for production (low latency, high throughput) - Tree for education (modular, demonstrates composition) - Python generator makes both easy (~zero extra cost) - Shows trade-off analysis

### 3.7.4 6.4 Why Python Generator vs Manual RTL?

**Decision:** Python-based code generation

**Rationale:** - Reuse existing APB crossbar automation (~95%) - Parameterization without manual edits - Consistent code quality - Educational value (code generation techniques) - Rapid design space exploration

---

## 3.8 7. Comparison to APB Crossbar

### 3.8.1 7.1 Similarities (~95% Code Reuse)

| Component | APB Crossbar | Delta (AXIS) | Reusable? |
|---|---|---|---|
| Request generation | Address decode | TDEST decode | [PASS] Pattern same |
| Per-slave arbitration | Round-robin | Round-robin | [PASS] Identical logic |
| Grant matrix | MxN grants | MxN grants | [PASS] Identical |
| Data multiplexing | Mux PRDATA | Mux TDATA/TVALID/TLAST | [PASS] Same pattern |
| Backpressure | PREADY | TREADY | [PASS] Renamed only |

### 3.8.2 7.2 Differences

| Aspect | APB Crossbar | Delta (AXIS) |
|---|---|---|
| **Address decode** | Range check (base/end) | Direct TDEST (simpler!) |
| **Packet concept** | Single transaction | Multi-beat packets with TLAST |
| **Arbitration** | Re-arbitrate every cycle | Lock until TLAST (packet atomicity) |
| **Signals** | PRDATA, PSLVERR | TDATA, TVALID, TLAST, TDEST, TID, TUSER |
| **Read/Write** | Separate paths | Unified data path (simpler!) |

### 3.8.3 7.3 Migration Effort

**Estimated Time:** ~75 minutes (from existing APB generator to AXIS)

**Tasks:** 1. Rename signals (10 min) 2. Simplify address decode to TDEST (5 min) 3. Add packet atomicity logic (15 min) 4. Add new signals (TLAST, TID, TUSER) to ports and mux (10 min) 5. Update module naming (5 min) 6. Test with 2x2 configuration (30 min)

---

## 3.9 8. Use Cases

### 3.9.1 8.1 Primary: RISC Cores + DSP Array

**Scenario:** 4 small RISC-V cores need to route compute tasks to 16 DSP accelerators

**Configuration:** - 4 masters (RISC cores) - 16 slaves (DSP arrays) - 64-bit data width - Flat topology (low latency critical)

**Benefits:** - Each RISC core can target any DSP (full flexibility) - Round-robin ensures fair DSP access - Low 2-cycle latency for task dispatch

### 3.9.2 8.2 Secondary: Educational Demonstration

**Scenario:** Teach students about interconnect design trade-offs

**Configuration:** - Generate flat crossbars (tree topology retired 2026-10-12) - Compare latency, throughput, resources - Show code generation techniques - Demonstrate verification methodology

**Benefits:** - Hands-on learning with working RTL - Clear performance comparisons - Demonstrates real-world design process

### 3.9.3 8.3 Future: Integration with RAPIDS

**Scenario:** Use Delta for compute fabric routing, RAPIDS for memory DMA

**Configuration:** - RAPIDS handles descriptor-based memory transfers - Delta routes compute tasks between processors - Protocol adapter (Network 2.0 <-> AXI-Stream)

**Benefits:** - Separate concerns (memory vs compute) - Reusable components - Scalable architecture

---

## 3.10 9. Verification Strategy

### 3.10.1 9.1 Generator Verification

**Approach:** 1. Unit tests for each generator function (Python unittest) 2. Lint generated RTL (Verilator) 3. Synthesis test (Vivado/Yosys) 4. Compare output for different parameters

**Success Criteria:** - All Python tests pass - Generated RTL lints clean - Synthesis completes without errors

### 3.10.2 9.2 RTL Verification

**Approach:** 1. CocoTB testbench framework (reuse AMBA patterns) 2. Test all MxS routing combinations 3. Concurrent traffic scenarios 4. Backpressure stress tests 5. Packet atomicity verification

**Success Criteria:** - 100% routing coverage - No deadlocks under stress - Packet atomicity confirmed - Performance matches model (±10%)

### 3.10.3 9.3 Model Validation

**Approach:** 1. Analytical model vs simulation model comparison 2. Simulation model vs RTL CocoTB results 3. Synthesis reports vs resource estimates

**Success Criteria:** - Latency within ±1 cycle - Throughput within ±15% - Resources within ±20%

---

## 3.11 10. Documentation Requirements

### 3.11.1 10.1 Specifications

**PRD** (this document) - Requirements, architecture, use cases

**Technical Specification** - Detailed block diagrams - Interface timing diagrams - Performance analysis

**User Guide** - Generator usage examples - Integration patterns - Best practices

### 3.11.2 10.2 Code Documentation

**Generator Code:** - Docstrings for all functions - Inline comments for complex logic - Usage examples in header

**Generated RTL:** - Module header with configuration - Block-level comments - Signal descriptions

**Performance Models:** - Algorithm explanations - Formula derivations - Validation methodology

---

## 3.12 11. Success Metrics

### 3.12.1 11.1 Functional Metrics

- [PASS] Generator produces lint-clean RTL
- [PASS] All routing combinations verified
- [PASS] No protocol violations
- [PASS] Packet atomicity enforced

### 3.12.2 11.2 Performance Metrics

- [PASS] Flat crossbar: 2-cycle latency
- [PASS] Tree topology: <=6-cycle latency
- [PASS] Throughput: >=0.7 transfers/cycle
- [PASS] Fmax: >=300 MHz

### 3.12.3 11.3 Quality Metrics

- [PASS] Code reuse: >=90% from APB generator
- [PASS] Model accuracy: ±10% vs RTL
- [PASS] Resource estimate: ±20% vs synthesis
- [PASS] Documentation: Complete and clear

---

## 3.13 12. Timeline and Milestones

### 3.13.1 Week 1: Generator and Models

- [PASS] Python generator (flat topology)
- [PASS] Analytical performance model
- [PASS] Simulation model (SimPy)
- [PASS] Specifications

### 3.13.2 Week 2: RTL and Verification

- Generate RTL variants (flat 4x16; tree variant retired 2026-10-12)
- CocoTB testbench framework
- Basic functional tests
- Performance validation

### 3.13.3 Week 3: Integration and Documentation

- RISC + DSP integration example
- Complete user guide
- Performance comparison report
- Educational materials

## 3.14 13. Open Questions

**Q1:** Should we add configurable FIFO insertion for burst buffering? - **Status:** Deferred to v1.1 (keep v1.0 simple)

**Q2:** Support for weighted round-robin arbitration? - **Status:** Deferred (default round-robin sufficient for now)

**Q3:** Integration with existing APB crossbar generator (unified tool)? - **Status:** Under consideration (separate for now, may merge later)

---

## 3.15 14. Revision History

| Version | Date | Author | Changes |
|---|---|---|---|
| 1.0 | 2025-10-18 | RTL Design Sherpa | Initial release |

## 3.16 16. Documentation

The aspirational DELTA mesh NoC specification was split out of this component on 2026-10-12 and lives at `projects/components/noc-ip/delta_spec/` (agent guide: `projects/components/noc-ip/CLAUDE.md`). To render it:

```
cd projects/components/noc-ip
./generate_pdf.sh --rev 0.25
```

Rendered DOCX/PDF land in `noc-ip/delta_spec/`. This component — the AXIS crossbar generator — is documented by README.md, QUICK_START.md, and `delta_index.md`.

## 3.17 Appendix A: Glossary

- **AXIS:** AXI-Stream (streaming variant of AXI protocol)
- **Delta:** Project name (river delta = branching flow, like crossbar routing)
- **Flat Topology:** Full MxN crossbar with all connections
- **Tree Topology:** Hierarchical composition of 1:2 and 2:1 nodes
- **Packet Atomicity:** Locking grant until TLAST (prevent interleaving)
- **TDEST:** Transaction destination (slave ID in AXI-Stream)
- **TID:** Transaction ID (master identifier in AXI-Stream)

---

**END OF PRD**

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 4 Claude Code Guide: Delta (AXI-Stream Crossbar Generator)

**Version:** 1.0 **Last Updated:** 2026-10-12 **Purpose:** AI-specific guidance for the Delta component (fabric-gen-ip)

---

## 4.1 Quick Context

**What:** Delta is a **Python-based AXI4-Stream (AXIS) crossbar generator**. It emits a single self-contained SystemVerilog module per configuration — a flat M×N stream crossbar routing `s_axis_*` masters to `m_axis_*` slaves.

**Status:** Active development. RTL generates and simulates; there is **no dv/ test suite yet** (the CocoTB suite is the known TODO). Re-homed from `noc-ip/delta` on 2026-10-12 — the move did not change any behavior.

**Your Role:** Help users generate, parameterize, and validate AXIS crossbars; keep generated RTL and this documentation in sync.

**Not this component:** the aspirational DELTA mesh NoC design draft (routers, virtual channels, XY routing) lives separately at `projects/components/noc-ip/delta_spec/` — **no NoC RTL exists**. When "delta" comes up without qualification, this crossbar generator is what is meant.

---

## 4.2 How It Actually Works (verified against `rtl/delta_axis_flat_4x16.sv`)

- **Route decode:** the target slave comes straight from `s_axis_tdest` (`DEST_WIDTH` bits) — there is **no address map**.
- **Arbitration:** per-slave round-robin; the grant is **packet-atomic** — once granted, a master holds the slave until `TLAST`.
- **Data path:** `TDATA/TVALID/TLAST/TDEST/TID/TUSER` crossbar mux is **combinational**; the **grant** is the registered stage.
- **Backpressure:** `TREADY` from the granted slave is steered back to the granted master only.
- **Optional flavors (2026-10-12, default off, default output byte-identical):** `TSTRB` (`--strb-width`) byte qualifiers, and AMBA 5 `TWAKEUP`/`TPARITY` (`--amba5`, `--parity-width` default DATA_WIDTH/8) so generated crossbars port-match `rtl/amba/axis4/*` and `rtl/amba/axis5/*`. TWAKEUP is TDEST-decoded (OR per slave) and may assert before TVALID, per the AMBA 5 hint semantics.
- **Optional:** packet counters.

Parameters on the generated module: `DATA_WIDTH`, `DEST_WIDTH`, `ID_WIDTH`, `USER_WIDTH`, `NUM_MASTERS`, `NUM_SLAVES`.

---

## 4.3 Repository Map

| Path | What |
|---|---|
| `bin/delta_generator.py` | The generator (`DeltaConfig`, `DeltaFlatCrossbar`, CLI) |
| `bin/delta_performance_model.py` | Analytical + simulation performance model |
| `bin/complete_tree_generator.py` | Retired tree-topology prototype (kept for history) |
| `rtl/delta_axis_flat_4x16.sv` | Reference generated crossbar (checked in) |
| `rtl_test/*.sv` | Generated variants (2×4, 3×8, 2×16, 4×16) |
| `rtl_test/delta_axis_flat_2x4_strb_amba5.sv` | Rich-flavor example (TSTRB + TWAKEUP/TPARITY) |
| `rtl/filelists/*.f` | Filelists (`$REPO_ROOT`-relative) |
| `README.md` / `PRD.md` / `QUICK_START.md` | Product docs (PRD REQ-GEN-002 is flat-only since 2026-10-12) |
| `delta_index.md` | Crossbar doc index |
| `TREE_TOPOLOGY_TEST_RESULTS.md` | Historical results for the retired tree path |

## 4.4 Critical Rules

1. **Flat only.** `--topology tree` (and `both`) prints "not yet implemented in framework version". Do not promise tree generation; REQ-GEN-002 was amended 2026-10-12.
2. **Regenerate, don't hand-edit.** `rtl/delta_axis_flat_4x16.sv` and the `rtl_test/` variants are generator outputs. Fix `bin/delta_generator.py` and re-emit; note `--topology tree` was retired — regenerate with `flat`.
3. **AXIS only.** No AXI4/AXI4-Lite/APB conversion here (PRD non-goal). For memory-mapped fabrics see siblings `../bridge/` and `../apbx-xbar/`.
4. **Filelist paths** in `rtl/filelists/*.f` are `$REPO_ROOT`-relative — keep them working from any cwd.
5. **Do not resurrect the NoC** (`noc-ip/delta_spec/`) unless the owner explicitly asks for NoC work.

## 4.5 Common Tasks

```
# Generate a fresh crossbar
python3 bin/delta_generator.py --topology flat --masters 4 --slaves 16
\
    --data-width 64 --output-dir rtl/

# Rich flavor: TSTRB + AMBA 5 sidebands (matches rtl/amba/axis4 +
axis5)
python3 bin/delta_generator.py --topology flat --masters 2 --slaves 4
\
    --data-width 32 --strb-width 4 --amba5 --output-dir rtl_test/

# Performance comparison (historical flat/tree data retained)
python3 bin/delta_performance_model.py --topology compare
```

---

**Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 5 Document Information

## 5.1 Delta Micro-Architecture Specification

| Property | Value |
|---|---|
| Document Title | Delta Micro-Architecture Specification |
| Version | 1.0 |
| Date | 2026-10-12 |
| Status | Released |
| Classification | Open Source - MIT License |

## 5.2 Revision History

| Version | Date | Author | Description |
|---|---|---|---|
| 1.0 | 2026-10-12 | RTL Design Sherpa | Initial release; describes the generated flat crossbar RTL including request decode, packet-atomic RR arbitration, mux, TREADY/TWAKEUP steering, counters, and assertions. |

## 5.3 Document Purpose

This Micro-Architecture Specification (MAS) is the internals document for the Delta AXI4-Stream crossbar generator — the one you open when you need to know what the generated RTL actually does, block by block. It covers:

- Block-level implementation of the generated SystemVerilog
- Arbitration state behavior and grant hold/release rules
- Cycle-level behavior examples
- Parameter-to-logic implications
- Verification strategy

For the high-level architecture, interface requirements, and integration guidelines, see the companion **Delta Hardware Architecture Specification (HAS)**.

## 5.4 Intended Audience

- RTL engineers reading or modifying the generated crossbar
- Verification engineers writing protocol checks
- System architects resolving timing or arbitration questions
- Technical reviewers auditing the implementation

## 5.5 Related Documents

- **Delta HAS** - Hardware Architecture Specification (high-level)
- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI assistant integration guide
- **QUICK_START.md** - Hands-on generator guide

RTL Design Sherpa · Learning Hardware Design Through Practice  Documentation Index · MIT License

---

# 6 Overview

## 6.1 What This Document Describes

This Micro-Architecture Specification (MAS) describes the internals of the SystemVerilog module produced by `bin/delta_generator.py`. It is intended for readers who need to understand, debug, or modify the generated RTL.

## 6.2 Generated Module Structure

The generator emits a single module named `delta_axis_flat_MxN` where `M` is `NUM_MASTERS` and `N` is `NUM_SLAVES`. The module contains:

1. **TDEST decode** — builds the per-slave request matrix.
2. **Per-slave round-robin arbiter** — packet-atomic grant logic.
3. **Crossbar mux** — combinational selection of payload and control.
4. **TREADY steering** — backpressure routing.
5. **TWAKEUP steering** — AMBA 5 wakeup hint routing (optional).
6. **Packet counters** — per-master and per-slave counters (optional).
7. **Assertions** — `ifdef FORMAL` checks.

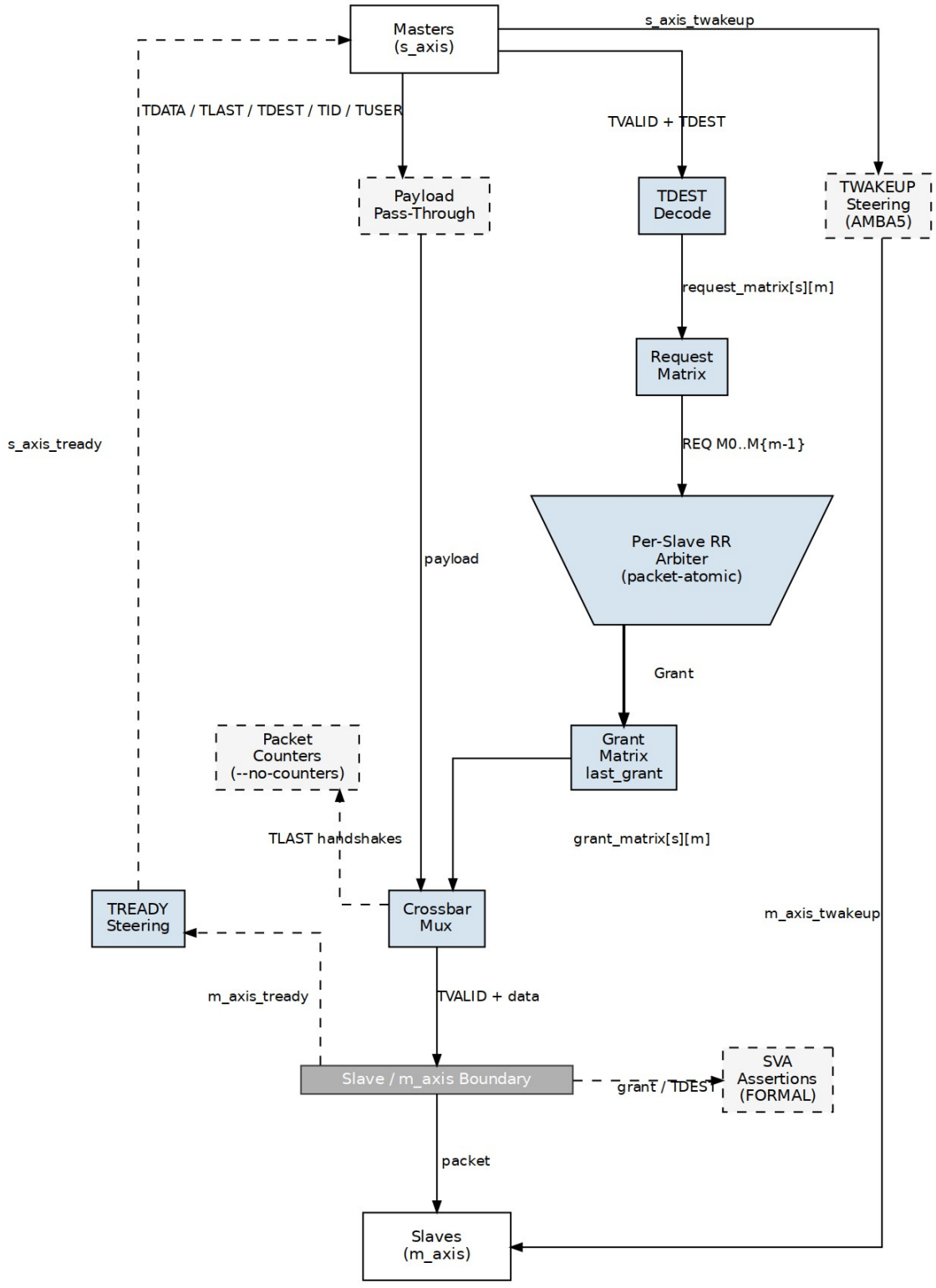
## 6.3 Key Micro-Architectural Decisions

*Table 1.1: Key Micro-Architectural Decisions*

| Decision | Rationale |
|---|---|
| Registered grant, combinational mux | Two-cycle latency with a short critical path. |
| Per-slave arbiter | Independent scheduling; no head-of-line blocking across slaves. |
| TDEST direct decode | No address map; simpler and lower latency than address-range decode. |
| Packet-atomic grant | Preserves AXI4-Stream packet semantics; prevents interleaving. |
| Optional sidebands default off | Keeps base output byte-identical to earlier versions. |

## 6.4 Block Diagram

### 6.4.1 Figure 1.1: Delta Micro-Architecture Block Diagram




*Delta MAS Block Diagram*

The micro-architecture diagram shows the internal blocks of the generated RTL. The request/control flow descends through TDEST decode, the request matrix, the packet-atomic round-robin arbiter (inverted-trapezium scheduler), the grant matrix, and the crossbar mux to the slave boundary. The payload flow follows its own route on the left. The backpressure flow returns up through TREADY steering as a dashed sideband. Optional blocks (TWAKEUP steering, packet counters, and SVA assertions) are dashed.

## 6.5 Reference RTL

The descriptions in this document are verified against:

- `rtl/delta_axis_flat_4x16.sv` (base flavor)
- `rtl_test/delta_axis_flat_2x4_strb_amba5.sv` (rich flavor)

RTL Design Sherpa · Learning Hardware Design Through Practice  Documentation Index · MIT License

---

# 7 Request Decode

## 7.1 Purpose

The request decode block translates each master's `s_axis_tvalid` and `s_axis_tdest` into a per-slave request matrix. A set bit in `request_matrix[s][m]` indicates that master `m` is requesting slave `s`.

## 7.2 Implementation

The decode is combinational:

```
logic [NUM_MASTERS-1:0] request_matrix [NUM_SLAVES];

always_comb begin
    for (int s = 0; s < NUM_SLAVES; s++) begin
        request_matrix[s] = '0;
    end
    for (int m = 0; m < NUM_MASTERS; m++) begin
        if (s_axis_tvalid[m] &&
            (s_axis_tdest[m] < NUM_SLAVES[$clog2(NUM_SLAVES+1)-1:0]))
begin
            request_matrix[s_axis_tdest[m]][m] = 1'b1;
        end
    end
end
```

## 7.3 TDEST Range Check

The comparison `s_axis_tdest[m] < NUM_SLAVES` prevents out-of-range indices from corrupting the `request_matrix` array. The width of the comparison is sized to cover the full `NUM_SLAVES` value.

## 7.4 Relationship to Arbitration

The request matrix is the input to each slave's round-robin arbiter. Only slaves with at least one request bit set are candidates for a new grant.

## 7.5 Notes

- The decode does not depend on `TREADY`; a master may request even while stalled.
- `TDEST` is sampled combinationally and must be stable while `TVALID` is asserted.
- There is no address map; `TDEST` is the slave index directly.

# 8 Per-Slave Round-Robin Arbiter

## 8.1 State Variables

Each slave `s` maintains three registered variables:

*Table 2.1: Arbiter State Variables*

| Variable | Width | Purpose |
|---|---|---|
| `grant_matrix[s]` | `NUM_MASTERS` bits | One-hot (or zero-hot) grant vector. |
| `last_grant[s]` | `clog2(NUM_MASTERS)` bits | Pointer to the most recently granted master. |
| `packet_active[s]` | 1 bit | Set while a packet is in progress on this slave. |

## 8.2 FSM Behavior

The arbiter has two operational states for each slave:

### 8.2.1 Idle State (`packet_active[s] == 0`)

- If `request_matrix[s]` is non-zero, perform round-robin arbitration.
- The search starts at `last_grant[s] + 1` and wraps through all masters.
- The first requesting master is granted; `last_grant[s]` is updated and `packet_active[s]` is set.
- If no master requests, `grant_matrix[s]` remains zero.

### 8.2.2 Active State (`packet_active[s] == 1`)

- The current grant is held.
- On the cycle where `m_axis_tvalid[s] && m_axis_tready[s] && m_axis_tlast[s]` is true, `packet_active[s]` is cleared and `grant_matrix[s]` is reset to zero.
- If `TLAST` is accepted, the next cycle may re-arbitrate for a new master.

## 8.3 Round-Robin Pseudocode

```
if (packet_active[s]) begin
    if (m_axis_tvalid[s] && m_axis_tready[s] && m_axis_tlast[s]) begin
        packet_active[s] <= 1'b0;
        grant_matrix[s]  <= '0;
    end
end else if (|request_matrix[s]) begin
    grant_matrix[s] = '0;
    for (int i = 0; i < NUM_MASTERS; i++) begin
        int m = (last_grant[s] + 1 + i) % NUM_MASTERS;
        if (request_matrix[s][m] && grant_matrix[s] == '0) begin
            grant_matrix[s][m] = 1'b1;
            last_grant[s]      = m[$clog2(NUM_MASTERS)-1:0];
            packet_active[s]   = 1'b1;
        end
    end
end else begin
    grant_matrix[s] <= '0;
end
```

## 8.4 Fairness

Round-robin guarantees that no requesting master is starved. The longest wait for a contending master is bounded by the number of other contending masters and the length of their packets.

## 8.5 Reset Behavior

On reset:

- `grant_matrix[s] <= '0`
- `last_grant[s] <= '0`
- `packet_active[s] <= 1'b0`

RTL Design Sherpa · Learning Hardware Design Through Practice  Documentation Index · MIT License

---

# 9 Crossbar Multiplexer

## 9.1 Purpose

The crossbar mux selects the payload and control signals from the granted master and drives them onto the selected slave interface. There is one mux per slave.

## 9.2 Implementation

Each slave mux is combinational:

```
generate
    for (genvar s = 0; s < NUM_SLAVES; s++) begin : gen_slave_mux
        always_comb begin
            m_axis_tdata[s]  = '0;
            m_axis_tvalid[s] = 1'b0;
            m_axis_tlast[s]  = 1'b0;
            m_axis_tdest[s]  = '0;
            m_axis_tid[s]    = '0;
            m_axis_tuser[s]  = '0;

            for (int m = 0; m < NUM_MASTERS; m++) begin
                if (grant_matrix[s][m]) begin
                    m_axis_tdata[s]  = s_axis_tdata[m];
                    m_axis_tvalid[s] = s_axis_tvalid[m];
                    m_axis_tlast[s]  = s_axis_tlast[m];
                    m_axis_tdest[s]  = s_axis_tdest[m];
                    m_axis_tid[s]    = s_axis_tid[m];
                    m_axis_tuser[s]  = s_axis_tuser[m];
                end
            end
        end
    end
endgenerate
```

## 9.3 Signal Selection

The mux forwards the following signals from the granted master to the selected slave:

- TDATA
- TVALID
- TLAST
- TDEST
- TID
- TUSER
- TSTRB (when enabled)
- TPARITY (when enabled)

## 9.4 Default Values

When no grant is active for a slave, the mux drives all outputs to zero and `m_axis_tvalid[s]` to low. This is a safe idle state.

## 9.5 Timing

Because the mux is combinational, the output settles after the grant and the input signals are stable. The grant is registered, so the mux select is stable for a full clock cycle once a packet begins.

# 10 TREADY Steering

## 10.1 Purpose

TREADY steering routes each slave's `m_axis_tready` back to the master that currently holds the grant for that slave. Ungranted masters see `s_axis_tready[m] == 0`.

## 10.2 Implementation

```
generate
    for (genvar m = 0; m < NUM_MASTERS; m++) begin : gen_master_tready
        always_comb begin
            s_axis_tready[m] = 1'b0;
            for (int s = 0; s < NUM_SLAVES; s++) begin
                if (grant_matrix[s][m]) begin
                    s_axis_tready[m] = m_axis_tready[s];
                end
            end
        end
    end
endgenerate
```

## 10.3 Behavior

- Only one slave can grant a given master at a time (packets are atomic and a master targets one slave per packet), so the loop effectively selects a single `m_axis_tready[s]`.
- If a master is not currently granted by any slave, it is stalled.
- Backpressure propagates combinationally from slave to master.

## 10.4 Deadlock Avoidance

The design avoids deadlock because:

- Grants are released after `TLAST`, freeing the slave for new requests.
- `TREADY` is combinationally steered; a slave ready transition immediately reaches the granted master.
- There are no FIFOs or credit loops in the base crossbar.

# 11 TWAKEUP Steering (AMBA5)

## 11.1 Purpose

When `--amba5` is enabled, the generated module includes `s_axis_twakeup` and `m_axis_twakeup` ports. `TWAKEUP` is an AMBA 5 hint that a master will soon send a transaction; it may assert before `TVALID`.

## 11.2 Implementation

The wakeup steering is combinational and independent of the grant:

```
always_comb begin
    for (int s = 0; s < NUM_SLAVES; s++) begin
        m_axis_twakeup[s] = 1'b0;
        for (int m = 0; m < NUM_MASTERS; m++) begin
            if (s_axis_twakeup[m] &&
                (s_axis_tdest[m] == s[$clog2(NUM_SLAVES)-1:0])) begin
                m_axis_twakeup[s] = 1'b1;
            end
        end
    end
end
```

## 11.3 Semantics

- `m_axis_twakeup[s]` is the OR of `s_axis_twakeup[m]` from every master whose `TDEST` points to slave `s`.
- No grant is required; the hint is decoded directly from `TDEST`.
- `TWAKEUP` may be high while `TVALID` is low.
- The hint is not a handshake; the slave is not required to respond before data arrives.

## 11.4 Integration Notes

- `TWAKEUP` is only emitted when `--amba5` is specified.
- The default parity width is `DATA_WIDTH / 8`; override with `--parity-width`.
- `TWAKEUP` does not affect arbitration timing.

# 12 Packet Counters

## 12.1 Purpose

Optional 32-bit packet counters track completed packets at the master boundary and the slave boundary. They are enabled by default and can be disabled with `--no-counters`.

## 12.2 Implementation

```
logic [31:0] pkt_count_master [NUM_MASTERS];
logic [31:0] pkt_count_slave [NUM_SLAVES];

generate
    for (genvar m = 0; m < NUM_MASTERS; m++) begin :
gen_master_counters
        always_ff @(posedge aclk or negedge aresetn) begin
            if (!aresetn)
                pkt_count_master[m] <= '0;
            else if (s_axis_tvalid[m] && s_axis_tready[m] &&
s_axis_tlast[m])
                pkt_count_master[m] <= pkt_count_master[m] + 1;
        end
    end

    for (genvar s = 0; s < NUM_SLAVES; s++) begin : gen_slave_counters
        always_ff @(posedge aclk or negedge aresetn) begin
            if (!aresetn)
                pkt_count_slave[s] <= '0;
            else if (m_axis_tvalid[s] && m_axis_tready[s] &&
m_axis_tlast[s])
                pkt_count_slave[s] <= pkt_count_slave[s] + 1;
        end
    end
endgenerate
```

## 12.3 Behavior

- A master counter increments when the master completes a packet (last beat accepted).
- A slave counter increments when the slave completes a packet (last beat accepted).
- Counters wrap at 32 bits.
- Counters are not exposed on module ports; they are internal observation points.

## 12.4 Disabling Counters

Use `--no-counters` to remove the counter logic and save area:

```
python3 bin/delta_generator.py --topology flat --masters 4 --slaves 16
\
    --data-width 64 --no-counters --output-dir rtl/
```

RTL Design Sherpa · Learning Hardware Design Through Practice  Documentation Index · MIT License

---

# 13 Assertions

## 13.1 Overview

All generated Delta modules include an `ifdef FORMAL` block with SystemVerilog assertions for debug and formal verification.

## 13.2 One-Hot Grant Check

Each slave must grant at most one master at a time:

```
assert property (@(posedge aclk) disable iff (!aresetn)
    $onehot0(grant_matrix[s])
) else $error("Grant matrix not one-hot for slave %0d", s);
```

`$onehot0` allows the grant vector to be all-zero (idle) but not multi-hot.

## 13.3 TDEST Range Check

Each master must drive a valid slave index when it asserts TVALID:

```
assert property (@(posedge aclk) disable iff (!aresetn)
    s_axis_tvalid[m] |-> s_axis_tdest[m] < NUM_SLAVES
) else $error("TDEST out of range for master %0d", m);
```

## 13.4 Enabling Assertions

Define `FORMAL` during simulation or formal analysis:

```
verilator --lint-only -DFORMAL rtl/delta_axis_flat_4x16.sv
```

## 13.5 Notes

- Assertions are disabled by default to avoid simulation overhead.
- The TDEST range assertion duplicates the combinational safety check in the request decode block.
- Additional protocol assertions (TVALID stability, TLAST packet integrity) are candidates for the planned cocotb testbench.

# 14 Multi-Master Contention

## 14.1 Scenario

Two masters, M0 and M1, both want to send packets to the same slave S0. S0 is initially idle. M0 sends a two-beat packet; M1 sends a one-beat packet.

## 14.2 Cycle Trace

Assumption: `last_grant[S0]` was 1 before the trace begins.

*Table 3.1: Contention Trace*

| Cycle | Event | request_matrix[S0] | grant_matrix[S0] | packet_active[S0] | Notes |
|---|---|---|---|---|---|
| 0 | M0 and M1 both request S0 | 2'b11 | 2'b00 | 0 | Slave idle. |
| 1 | Arbiter picks M0 (RR after last_grant=1) | 2'b11 | 2'b01 | 1 | Grant registered. |
| 2 | M0 beat 0 accepted | 2'b10 | 2'b01 | 1 | M1 waits. |
| 3 | M0 beat 1 (TLAST) accepted | 2'b10 | 2'b00 | 0 | Packet ends; grant released. |

| Cycle | Event | request_matrix[S0] | grant_matrix[S0] | packet_active[S0] | Notes |
|---|---|---|---|---|---|
| 4 | Arbiter picks M1 | 2'b10 | 2'b10 | 1 | M1 now granted. |
| 5 | M1 beat 0 (TLAST) accepted | 2'b00 | 2'b00 | 0 | Both packets complete. |

## 14.3 Observations

- M1 waits while M0 completes its packet.
- The arbiter does not re-arbitrate inside a packet.
- After TLAST, the next arbitration cycle grants M1 because M0 was the previous grant.
- Round-robin ensures M1 is not starved.

## 14.4 Same-Cycle Release and Re-Arbitration

On the cycle that TLAST is accepted, the grant is released. The next rising edge can register a new grant if a request is pending. This gives a one-cycle gap between back-to-back packets from different masters.

# 15 Packet Interleave Prevention

## 15.1 Rule

A slave can source data from only one master within a single packet. The grant is locked from the first beat of the packet until the TLAST beat is accepted.

## 15.2 Mechanism

The `packet_active[s]` flag is set when a new grant is made. While `packet_active[s]` is high:

- `grant_matrix[s]` is held constant.
- The crossbar mux continues to select the same master.
- `s_axis_tready` for the granted master follows only `m_axis_tready[s]`.

When `m_axis_tvalid[s] && m_axis_tready[s] && m_axis_tlast[s]` is true, `packet_active[s]` clears and the grant is released.

## 15.3 Why It Matters

AXI4-Stream slaves often expect packet-atomic streams. If beats from different masters were interleaved, a slave receiving the mixed stream could misinterpret packet boundaries, corrupt state, or violate protocol.

## 15.4 Example: Attempted Interleave

Suppose M0 and M1 both target S0 and both have multi-beat packets:

*Table 3.2: Interleave Prevention*

| Cycle | Without atomicity (hypothetical) | With Delta packet atomicity |
|---|---|---|
| 0 | M0 beat 0 | M0 beat 0 |
| 1 | M1 beat 0 (interleaved!) | M0 beat 1 |
| 2 | M0 beat 1 | M0 beat 2 (TLAST) |
| 3 | M1 beat 1 (TLAST) | M1 beat 0 |

Delta prevents the interleaved case by holding M0's grant until its `TLAST`.

## 15.5 Corner Cases

- If the granted master drops `TVALID` mid-packet, the slave sees no valid beats until the master reasserts it. The grant is still held.
- If the downstream slave is never ready, the packet can stall indefinitely; there is no timeout. This is consistent with standard AXI4-Stream behavior.



---

# 16 Module Structure

## 16.1 Generated File

The generator produces one file per configuration:

`delta_axis_flat_MxN.sv`

where `M` is `NUM_MASTERS` and `N` is `NUM_SLAVES`.

## 16.2 Module Header

The module begins with parameters and port declarations. The exact port list depends on the selected richness options.

### 16.2.1 Base Flavor Header

```
module delta_axis_flat_4x16 #(
    parameter int DATA_WIDTH  = 64,
    parameter int DEST_WIDTH  = 4,
    parameter int ID_WIDTH    = 2,
    parameter int USER_WIDTH  = 1,
    parameter int NUM_MASTERS = 4,
    parameter int NUM_SLAVES  = 16
)(
    input  logic aclk,
    input  logic aresetn,
    // s_axis and m_axis base ports...
);
```

### 16.2.2 Rich Flavor Additions

When `--strb-width` is used, `STRB_WIDTH` is added to the parameter list and `s_axis_tstrb`/`m_axis_tstrb` are added to the ports.

When `--amba5` is used, `PW_WIDTH` is added to the parameter list and `s_axis_twakeup`/`s_axis_tparity`/`m_axis_twakeup`/`m_axis_tparity` are added to the ports.

## 16.3 Internal Organization

Inside the module, the generator emits sections in this order:

1. Header comment with configuration.
2. Request decode.
3. Per-slave round-robin arbiter.
4. Crossbar data mux.
5. TREADY steering.
6. TWAKEUP steering (if enabled).
7. Performance counters (unless disabled).
8. Assertions (under `ifdef FORMAL`).

## 16.4 Filelist

The checked-in RTL includes `$REPO_ROOT`-relative filelists under `rtl/filelists/` for use with simulation and synthesis flows.

RTL Design Sherpa · Learning Hardware Design Through Practice  Documentation Index · MIT License

---

# 17 Parameters and Logic Implications

## 17.1 Parameter Summary

*Table 4.1: Parameter-to-Logic Mapping*

| Parameter | Generated From | Logic Implication |
|---|---|---|
| DATA_WIDTH | --data-width | Width of all TDATA and TPARITY default calculations. |
| DEST_WIDTH | clog2(NUM_SLAVES) | Width of TDEST; determines request decode comparison width. |
| ID_WIDTH | clog2(NUM_MASTERS) | Width of TID; pass-through only. |
| USER_WIDTH | --user-width | Width of TUSER; pass-through only. |
| NUM_MASTERS | --masters | Size of s_axis_* arrays and grant/request matrix columns. |
| NUM_SLAVES | --slaves | Size of m_axis_* arrays and grant/request matrix rows. |
| STRB_WIDTH | --strb-width | Width of TSTRB; adds mux ports when > 0. |
| PW_WIDTH | --parity-width or DATA_WIDTH/8 | Width of TPARITY; adds mux ports when --amba5 is set. |

## 17.2 Width Calculations

The generator computes:

```
dest_width = max(1, ceil(log2(num_slaves)))
id_width   = max(1, ceil(log2(num_masters)))
```

A single master or slave still gets a 1-bit width to avoid zero-width signals.

## 17.3 Parameter Interaction

- `STRB_WIDTH == 0` omits `TSTRB` entirely.
- `PW_WIDTH` is meaningful only when `--amba5` is set; otherwise `TPARITY` and `TWAKEUP` ports are not generated.
- `--parity-width` without `--amba5` is rejected by the CLI parser.

## 17.4 RTL Constants

The generator uses `generate`/`genvar` loops sized by `NUM_MASTERS` and `NUM_SLAVES`. Changing these parameters after generation requires regenerating the module; the parameter is set at generation time, not truly configurable post-generation. This is intentional: it allows the synthesizer to optimize the fixed-size structure.

# 18 Test Strategy

## 18.1 Current Status

As of 2026-10-12, **Delta has no dv/ test suite**. The RTL is lint-clean and the generator output matches checked-in reference files, but a cocotb-based verification environment is the known TODO.

## 18.2 Recommended Test Plan

When a testbench is created, it should cover the following areas.

### 18.2.1 1. Routing Coverage

- For every master `m` and every slave `s`, send at least one packet from `m` to `s`.
- Verify that only the target slave receives the packet.
- Verify that `m_axis_tdest[s]` matches the sent `TDEST`.

### 18.2.2 2. Arbitration Fairness

- Have multiple masters send continuous packets to the same slave.
- Verify that each master receives approximately equal bandwidth over time.
- Verify that no master is starved.

### 18.2.3 3. Packet Atomicity

- Have two masters send multi-beat packets to the same slave.
- Verify that slave receives the first packet's beats contiguously up to `TLAST` before any beat of the second packet.

### 18.2.4 4. Backpressure

- Randomly deassert `m_axis_tready[s]`.
- Verify that only the granted master stalls and that no data is lost.

### 18.2.5 5. Reset

- Assert `aresetn` mid-transaction.
- Verify all grants clear and counters reset.

### 18.2.6 6. Rich Flavor

- With `--strb-width`, verify `TSTRB` pass-through.
- With `--amba5`, verify `TWAKEUP` OR-per-slave and `TPARITY` pass-through.

### 18.2.7 7. Formal Assertions

- Run with `+define+FORMAL` or equivalent.
- Verify the one-hot grant and TDEST-range assertions never fire.

## 18.3 Lint and Generation Checks

Until a testbench exists, the verification consists of:

```
python3 bin/delta_generator.py --topology flat --masters 4 --slaves 16
\
    --data-width 64 --output-dir rtl/
verilator --lint-only rtl/delta_axis_flat_4x16.sv
```